CS 300 – Chapter 6 Notes

Trees

Jose de Lima

April 21, 2026

Key idea Chapter map

Where this sits in CS 300. Chapter 4 built *linear* structures: one pointer per node, one successor each. Chapter 6 adds the branching factor and introduces **hierarchical** storage. The central artifact is the **binary search tree** (BST) – a shape that can do insert, find, and remove in $O(\log n)$ when balanced, and gives you **sorted iteration** for free.

What you add to your toolkit here:

- **Tree vocabulary**: root, leaf, parent, child, depth, height, path, level, full vs. complete vs. perfect.
- **Binary tree** structure and the three canonical **traversals**: pre-order (root, L, R), in-order (L, root, R – sorted on a BST), post-order (L, R, root).
- **BST invariant**: every node's left subtree has smaller keys, right subtree has larger keys.
- **BST insert** (walk down, place at null) and **BST search** ($O(h)$, halving at each step in the best case).
- **BST removal** in its four cases: leaf, left-only child, right-only child, two children (replace with in-order successor / predecessor).
- **Height vs. balance**: a balanced tree gives $h = O(\log n)$; an unbalanced one degenerates to a linked list ($h = n$). Motivates AVL / red-black trees (ch. 7 onward).
- **Implementation variants**: with explicit parent pointers vs. without; recursive vs. reference-to-pointer.

Mastery checklist – you should be able to, without references:

1. Define the BST invariant and state why it depends on both subtrees, not just one.
2. Walk down a BST to find a key, and say why each step is a comparison and a pointer dereference.
3. Insert a sequence of keys into an initially empty BST by hand, drawing the tree after each step.
4. Write all three traversals iteratively *and* recursively; know which one on a BST produces sorted output.
5. Remove a node in a BST under each of the four cases; explain why the two-children case uses the in-order successor.
6. Argue when BST ops are $O(\log n)$ vs. $O(n)$; give a concrete insertion order that degenerates the tree.
7. Compare BST to `std::unordered_map` and pick the right container for (a) point lookup only, (b) range queries, (c) sorted iteration.

Looking ahead. The whole *week 7+ / ch. 7–9* arc is about making the BST $O(\log n)$ guarantee *tight* rather than best-case-only: AVL trees enforce balance via rotations, red-black trees relax it for faster

inserts, 2-3-4 trees generalize to B-trees on disk. Every one of those chapters assumes total fluency with vanilla BST from here – so if any of the mastery checklist feels shaky, this is the chapter to over-practice.

1 6.1 Binary Trees: Vocabulary and Shape

Trees are the first *hierarchical* data structure in this course. Chapters 3–4 built linear structures: every node had at most one successor, so iteration was just a walker loop. Trees branch. That single change — two children instead of one next — reshapes everything: traversal, cost analysis, and the class of problems you can solve efficiently.

Definition — Binary tree

A hierarchical data structure in which each node has at most two children: a *left child* and a *right child*. Either child may be absent. The tree has a distinguished *root* node (no parent) and every other node has exactly one parent.

The core vocabulary

This section is a dictionary. Memorize these words — every later section uses them, often without redefinition.

Term	Meaning
Root	The unique node with no parent
Leaf	A node with no children
Internal node	A node with at least one child
Parent	A node's direct ancestor (the node one step toward root)
Child	A node's direct descendant
Edge	The link from parent to child
Depth of a node	Number of edges on the path from root to the node. Root has depth 0
Level	Set of all nodes at the same depth
Height of the tree	Maximum depth of any node. A single-node tree has height 0
Ancestors	All nodes on the path from a node up to the root
Descendants	All nodes reachable by following child edges
Subtree	A node plus all of its descendants, viewed as a tree in its own right

Key idea Trees are recursive by nature

Notice the subtree definition. Every node in a binary tree is itself the root of a binary tree — the subtree rooted at that node. This recursive structure is why tree algorithms are almost always written recursively: "do X to the left subtree, do X to the right subtree, combine the results." The iterative versions exist but are awkward. Get comfortable with recursion on trees.

Visualizing the vocabulary

Consider this tree:

```
    A       (root, depth 0)
           / \
  B   C     (depth 1, level 1)
           / \\
D   E   F   (depth 2, level 2)
```

A is the root. D, E, F are leaves (no children). B and C are internal nodes. The path A → C → F has length 2, so F has depth 2. The tree's height is 2 (the deepest leaf). A is an ancestor of D, E, and F. The subtree rooted at B contains B, D, E.

Three special shapes

Some binary trees have names for their particular structure. These show up in algorithm analysis as best or worst cases.

Shape	Definition
Full	Every node has either 0 or 2 children (never exactly 1)
Complete	Every level is filled, *except possibly the last*, and the last level's nodes are packed to the left
Perfect	Every internal node has 2 children *and* every leaf is at the same depth

A perfect tree is both full and complete. A complete tree is not necessarily full (the last-level-partially-filled case). A full tree is not necessarily complete.

Key idea Why "complete" is the most useful shape

Complete binary trees have a magical property: they can be stored in an array with no wasted slots, using simple index arithmetic (children of index i are at $2i + 1$ and $2i + 2$). This is the basis of the binary heap, which backs `std::priority_queue` and heapsort. Complete trees are where arrays and trees merge: you get the structure of a tree and the memory layout of an array.

Why height matters so much

Almost every tree operation's cost is $O(\text{height})$, not $O(n)$. A search walks from root to leaf; an insert follows the same path. If the tree is *balanced*, the height is $O(\log n)$ — so operations are logarithmic. If the tree degenerates into a chain (every node has only one child), the height is $n - 1$, and you've reinvented the linked list, poorly.

Shape	Height	Cost of one search
Balanced (bushy)	$O(\log n)$	$O(\log n)$
Worst case (zigzag chain)	$O(n)$	$O(n)$

For $n = 10^6$: $\log_2 n \approx 20$. The difference between 20 operations and a million is the entire reason tree-based structures exist.

Gotcha Unbalanced trees are the killer bug

Every asymptotic guarantee for trees in this chapter (BST search, insert, remove) assumes the tree stays balanced. Inserting sorted input into a naive BST produces a worst-case chain. Section 6.9 introduces AVL trees, which preserve balance automatically. Until then, be aware that the $O(\log n)$ claim has an unstated precondition.

Node representation in C++

Example Binary tree node

```
template <typename T>
struct TreeNode {
    T          data;
    TreeNode*  left  = nullptr;
    TreeNode*  right = nullptr;
};
```

Compare with the DLL node from chapter 4: same "three fields," but instead of `prev`/`next` pointing to siblings in a linear order, we have `left`/`right` pointing to children in a branching hierarchy. The memory layout is identical; the semantics are different.

Aside Modern C++: consider `unique_ptr`

Hand-managing parent-owns-children pointers with `new` and `delete` is error-prone — you must write a recursive destructor, a copy constructor, etc. Using `std::unique_ptr<TreeNode> left, right;` makes ownership automatic: when a node is destroyed, its `unique_ptr` members destroy their children, which destroy theirs, recursively. The whole subtree cleans itself up. We'll use raw pointers for pedagogical clarity through this chapter; in real code, prefer smart pointers.

Key idea Chapter 6 roadmap

This chapter builds up in layers. First: shape and vocabulary (this section). Then: traversal patterns (6.2). Then: binary search trees (6.3–6.8), which use ordering to make search $O(\log n)$ on average. Then: balanced BSTs (AVL, red-black in 6.9–6.10), which guarantee $O(\log n)$ even in the worst case. The theme is "use structure (ordering, balance) to eliminate worst-case behavior."

2 6.2 Why Trees Matter: Applications

Before the algorithms, a reality check: what are trees *for*? Lists and arrays are obvious — things that come one after another. Trees represent a different kind of data entirely, and the examples are everywhere once you know to look.

Key idea Trees model hierarchy

Any time the data has a "contains" or "is part of" relation, a tree fits. Directories contain files. HTML elements contain elements. Class hierarchies inherit. Organizations have managers. The real world is full of parent–child relationships, and trees are the canonical data structure for them.

1. File systems

The filesystem on your laptop is a tree. The root is `/` (Unix) or `C:\` (Windows). Each directory is an internal node; each file is a leaf.

```
/
|- etc/
|   |- passwd            (leaf)
|   \- hosts            (leaf)
\- home/
    |- jose/
    |   \- notes.tex (leaf)
    \- guest/        (empty dir, leaf)
```

Filesystem trees are *not* binary — a directory can contain any number of entries. But many file-system internals (like B-trees for directory indexes) *are* balanced trees, just with higher fanout.

2. Expression / parse trees

Compilers parse `3 + 4 * 5` into a tree:

```
  +
 / \
3  *
  / \
 4  5
```

Evaluation is post-order traversal (6.3): compute children first, then apply the operator. Every compiler, every calculator app, every SQL query planner uses an expression tree at some point.

3. DOM, HTML, XML

A web page is a tree. `<html>` contains `<head>` and `<body>`; each contains more elements; etc. When JavaScript calls `document.querySelector`, it is walking a tree. When the browser renders CSS, it is traversing a tree. The "Document Object Model" is literally a tree.

4. Binary space partitioning (BSP) in graphics

Video games use BSP trees to avoid rendering things off-screen. The game world is recursively split: left half vs. right half, then each quadrant, then each octant, etc. Each interior node of the tree owns a region; each leaf owns a small cell with a handful of objects. To figure out what's visible, the renderer walks the tree from root to leaves, pruning entire branches whose regions don't intersect the camera frustum. Thousands of objects become a $O(\log n)$ lookup.

5. Search-structured trees

The rest of this chapter focuses on this category. When the data is a searchable collection, a *search tree* (BST, AVL, red-black, B-tree) gives you $O(\log n)$ operations — slower than a hash table in the average case, but with two huge advantages:

- **Ordered iteration**: walk the tree in order to produce sorted output. Hash tables can't.
- **Range queries**: "give me everything between x and y" is fast. Hash tables can't do this at all without scanning.

This is why `std::map` and `std::set` exist alongside `std::unordered_map` and `std::unordered_set`. If you need ordering or ranges, you use the tree-based version.

6. Priority queues and heaps

A binary heap (a specific kind of complete binary tree) is the standard implementation of a priority queue. Operating systems use one to schedule processes; Dijkstra's shortest-path algorithm uses one to pick the next vertex; event-driven simulations use one for the event queue. Section 6.7 covers heaps directly.

7. Decision trees and tries

Decision trees are the standard data structure in machine learning for if-then-else rule chains. *Tries* (prefix trees) store strings so that prefix lookups are $O(\text{length})$ regardless of dictionary size — used in autocomplete, spell checkers, and IP routing tables.

A quick summary

Domain	Tree use	Payoff
OS / storage	File system, DB index (B-tree)	Hierarchical naming, $O(\log n)$ lookup
Compilers	Parse / AST	Structure-preserving evaluation
Web / UI	DOM	Inherent nesting
Graphics / gaming	BSP, quadtree, octree	Spatial pruning
Containers	`std::map`, `std::set`	Ordered, range queries
Scheduling	Heap / priority queue	$O(\log n)$ min/max
ML / text	Decision tree, trie	Hierarchical decisions, prefix search
Networking	Longest-prefix trie	IP routing

Aside Trees are the "default" structure for many problems

If your data has natural ordering *and* you need range queries, go tree. If it has a natural parent–child relation, go tree. If you need both random access and ordered traversal, go tree. The hash table is faster for pure lookup, but it's a specialist — the tree is a generalist.

Key idea The rest of chapter 6

The remaining sections focus on **binary search trees** — one specific tree shape where the data is *sorted* into the structure. That ordering is what gets you $O(\log n)$ search. Then we'll see what goes wrong when the tree gets unbalanced, and how AVL trees fix it. The applications surveyed here are the "why"; the upcoming sections are the "how."

3 6.3 Binary Search Trees (BST)

A *binary search tree* adds one constraint to the binary tree: for every node, everything in the left subtree is smaller and everything in the right subtree is larger. That single ordering invariant is what turns a tree into a *searchable* structure with $O(\log n)$ lookup on average.

Definition — Binary Search Tree (BST)

A binary tree with the *BST property*: for every node N, all keys in N's left subtree are $\leq N$'s key, and all keys in N's right subtree are $\geq N$'s key. The property must hold for *every* node, not just the root — that recursion is the key point.

The ordering property

Here are two small trees with keys 10, 20, 30. Only one is a BST:

```
    20                  10
   /  \                /  \
  10  30              30  20
```

BST: left $\leq$ root $\leq$ right | Not a BST: 30 is in 10's left subtree but > 10

The property must hold *recursively*. It's not enough to check root $\leq$ left-child and root $\geq$ right-child — you have to check that *every* key in the left subtree is $\leq$ the root, not just the direct left child.

Gotcha The classic BST validation bug

A naive "valid BST" check looks at each node and its immediate children. This misses cases like:

```
    20
   /  \
  10  30
       \
       15
```

15 is < 20 (the root) but it's in 20's *right* subtree. Local validity ($30 > 15$) doesn't catch it. The correct check threads min/max bounds down the recursion — each subtree must satisfy its inherited range, not just its parent.

Search: the payoff

Given the BST property, searching for a key is almost identical to binary search on a sorted array — you compare, then go left or right.

Example BST search

```
TreeNode* search(TreeNode* root, int key) {
    TreeNode* cur = root;
    while (cur) {
        if      (key == cur->data) return cur;
        else if (key <  cur->data) cur = cur->left;
        else                       cur = cur->right;
    }
    return nullptr;  // not found
}
```

Each step eliminates an entire subtree. Starting at the root, you either find the key or descend by one level. The cost is $O(h)$ where h is the tree's height — not $O(n)$.

The height is everything

For a *balanced* tree (all levels full or nearly so): $h = \lfloor \log_2 n \rfloor$. For $n = 10^6$, that's 19. A linear search would take a million comparisons; a balanced BST takes fewer than twenty.

Nodes	Min height	Worst-case height
10	3	9
100	6	99
1,000	9	999
1,000,000	19	999,999
1,000,000,000	29	$10^9 - 1$

The "worst-case height" column is what happens if you insert keys in sorted order: each new key becomes the right child of the previous, and the tree degenerates into a linked list. That is the BST's failure mode, and the reason for balanced variants (AVL in 6.9).

Successors and predecessors

The BST property induces an ordering on the nodes. A node's *successor* is the next key in sorted order; its *predecessor* is the previous one.

Case	How to find the successor of node N
N has a right subtree	The leftmost node of the right subtree (go right once, then left until you can't)
N has no right subtree	The nearest ancestor of N whose left subtree contains N (walk up until you go up-and-to-the-right)

This matters because an *in-order traversal* of the BST (left, then root, then right — section 6.4) visits nodes in sorted order. The successor operation is the building block of any iterator over a BST.

Key idea A BST is binary search with mutability

Binary search on a sorted array: $O(\log n)$ lookup, but insert is $O(n)$ because every insert shifts elements. A BST: $O(\log n)$ lookup *and* $O(\log n)$ insert/remove (when balanced). That combination — fast search *and* fast modification — is why BSTs exist. Sorted arrays are great for read-heavy workloads; BSTs are great for mixed read/write.

Duplicates?

The definition says "left $\leq$ root $\leq$ right," which technically allows duplicates on either side. Implementations differ:

- **Store once, count multiplicities**: each node has a count. Insert of existing key just increments.
- **Allow duplicates on one side**: always go left (or always right) when equal. Keeps the tree a valid BST but can mildly skew height.
- **Reject duplicates**: `insert` is an upsert or returns "already exists." `std::set` takes this route; `std::multiset` doesn't.

For the rest of this chapter we'll mostly assume unique keys; it's the simpler mental model.

Aside STL: `std::set` / `std::map`

Both are almost always implemented as *red-black trees*, which are a balanced-BST variant with stronger guarantees than AVL for mutation workloads. They provide $O(\log n)$ insert, find, erase, and support `lower_bound` / `upper_bound` for range queries. When you reach for `std::map` in real code, you are

getting a BST whose balance is managed for you.

Key idea Where this chapter goes next

6.4 covers tree traversals — the three "visit every node" orderings that unlock most BST algorithms. 6.5–6.6 cover insertion and removal, including the tricky two-children deletion case. 6.7 introduces heaps (a different tree shape for a different purpose). 6.8 does BST cost analysis formally. 6.9– 6.10 solve the unbalanced-tree problem with self-balancing variants. Each section adds one more piece to a complete picture of "how a std::map actually works."

4 6.4 BST Search, Closer Look

Section 6.3 introduced BST search in passing. This section looks at it in detail — the algorithm in its iterative and recursive forms, the comparison with binary search on arrays, and the subtle points that trip people up.

The algorithm, stated precisely

Example Iterative BST search

```
TreeNode* search(TreeNode* root, int key) {
    TreeNode* cur = root;
    while (cur != nullptr) {
        if      (key == cur->data) return cur;
        else if (key <  cur->data) cur = cur->left;
        else                       cur = cur->right;
    }
    return nullptr;
}
```

The loop invariant: if the key is in the tree, it's in the subtree rooted at `cur`. Each comparison either finds the key or shrinks the candidate subtree. When `cur` becomes `nullptr`, you've walked off the tree — the key is definitely not present.

The recursive version

Example Recursive BST search

```
TreeNode* search(TreeNode* node, int key) {
    if (!node)              return nullptr;
    if (key == node->data)  return node;
    if (key <  node->data)  return search(node->left,  key);
    else                    return search(node->right, key);
}
```

Four lines, three cases, one base case. The recursive version reads more like the definition of BST search but uses $O(h)$ stack frames — the iterative version is $O(1)$ space. Most production implementations go

iterative for that reason.

Key idea Binary search on an array vs. a BST

They are the *same algorithm*. Both halve the candidate set at each step. The difference is where the "halving" structure lives: in the array, it's the implicit "middle index" at each recursion; in the BST, it's the explicit parent-child pointer layout built into the data. The tree *caches* the binary- search decision structure in its shape.

Cost analysis

Tree shape	Search cost	Notes
Perfectly balanced	$O(\log n)$	Each step halves the subtree
Randomly built	$O(\log n)$ expected	Average-case analysis
Completely skewed (chain)	$O(n)$	Linked list by another name

The second row is the interesting one. If you insert n keys in uniformly random order into a BST, the expected height is $O(\log n)$ — so even without explicit balancing, randomly- ordered inserts give you good performance. The failure mode is *structured* input: sorted, reverse-sorted, or near-sorted data, all of which produce skewed trees.

Gotcha Real-world data is rarely random

"Expected $O(\log n)$" sounds reassuring, but real inputs are often ordered: timestamps, sequential IDs, alphabetized names. If you plan to use a BST for user-supplied or database data, assume the worst case. Use a self-balancing variant (AVL, red- black) rather than a raw BST. The extra bookkeeping is a small constant-factor cost for worst-case $O(\log n)$ guarantees.

What "not found" looks like

The algorithm's not-found behavior falls out naturally: you walk left/right until you hit `nullptr`. The last non-null node you visited is the node where the key *would be inserted* if you were adding it. This is useful for `insert` (6.5): you reuse the search path to find the insertion point.

Example Search-plus-parent, useful for insertion/removal

```
// Returns the found node, or nullptr.
// Also writes the parent of the search's last position to *outParent.
TreeNode* searchWithParent(TreeNode* root, int key, TreeNode** outParent)
    {
    TreeNode* cur    = root;
    TreeNode* parent = nullptr;
    while (cur) {
        if (key == cur->data) { *outParent = parent; return cur; }
        parent = cur;
        cur    = (key < cur->data) ? cur->left : cur->right;
    }
    *outParent = parent;
    return nullptr;  // parent is where to attach a new node
}
```

This variant is the workhorse for insert and remove operations in upcoming sections — you need the parent to splice in or unsplice from the tree.

Comparing BST search to other structures

Structure	Search (avg)	Search (worst)	Ordered iteration?
Linked list	$O(n)$	$O(n)$	Yes (insertion order)
Sorted array	$O(\log n)$	$O(\log n)$	Yes
Hash table	$O(1)$	$O(n)$	No
BST (unbalanced)	$O(\log n)$	$O(n)$	Yes (in-order)
BST (balanced)	$O(\log n)$	$O(\log n)$	Yes (in-order)

Key idea When to pick a BST over a hash table

Hash tables are faster for raw point lookups ($O(1)$ vs. $O(\log n)$), so if all you need is "is this key here, yes or no?", pick a hash table. Pick a BST when you need *ordered* operations: find-min, find-max, find-successor, find-predecessor, range queries ("everything between 50 and 100"), or iteration in sorted order. Hash tables can't do any of those without $O(n \log n)$ postprocessing.

Aside BSTs in STL, again

`std::set<T>::find` is $O(\log n)$ worst case because the underlying red-black tree is balanced. `std::set::lower_bound(k)` returns an iterator to the smallest key $\geq k$ — also $O(\log n)$. This is exactly "search with parent" turned into a user-facing API, and it's the main reason people pick `std::set` over `std::unordered_set`.

5 6.5 BST Insertion

Inserting into a BST is *search plus one write*. You walk the tree exactly as you would to search for the new key; when you hit `nullptr` — the slot where the key "would be" — you attach the new node there. That's it. The BST property is preserved automatically because you followed it on the way down.

Definition — BST insert

Given a BST and a new key, the insert operation attaches a new leaf node at the unique position that preserves the BST property. The insertion point is found by simulating a search for the new key: it is always the first `nullptr` child pointer encountered along the comparison path.

The algorithm

Example BST insert

```
void insert(TreeNode*& root, int key) {
    if (!root) { root = new TreeNode{key, nullptr, nullptr}; return; }

    TreeNode* cur = root;
    while (true) {
        if (key < cur->data) {
```

```
            if (!cur->left)  { cur->left = new TreeNode{key, nullptr,
                nullptr}; return; }
            cur = cur->left;
        } else {
            if (!cur->right) { cur->right = new TreeNode{key, nullptr,
                nullptr}; return; }
            cur = cur->right;
        }
    }
}
```

Two observations about the code:

- `TreeNode*&` (reference to pointer) in the signature lets the empty-tree case assign to the caller's root pointer. Without the reference, you'd return the new root and require the caller to reassign.
- New nodes are always **leaves**. A BST never inserts in the middle of the tree — only at the frontier where a `nullptr` existed.

Walk-through

Start with this tree and insert 35:

```
      50
    /   \
  30    70
    /\
  20 40
```

1. At root 50: $35 < 50$, go left.
2. At 30: $35 > 30$, go right.
3. At 40: $35 < 40$, check left. Left is null — attach 35 here.

Result: 35 becomes 40's left child. The BST property holds at every ancestor ($30 < 35$ in its right subtree, $50 > 35$ in its left subtree, $40 > 35$ as its direct parent).

Cost analysis

Tree shape	Insert cost	Notes
Balanced	$O(\log n)$	Height matches search
Random inserts	$O(\log n)$ expected	Average-case
Sorted inserts (worst)	$O(n)$	Tree becomes a chain

Space is $O(1)$ for the search plus $O(1)$ for the new node — no extra bookkeeping.

Gotcha Sorted input destroys a raw BST

Insert keys `1, 2, 3, 4, 5` in that order into an empty BST. You get:

```
1
```

```
\
2
 \
 3
  \
  4
   \
   5
```

A right-leaning chain. Search is $O(n)$, not $O(\log n)$. This is why you must either (1) shuffle your input before inserting, or (2) use a self-balancing BST (6.9).

Recursive version

Example Recursive insert

```
TreeNode* insert(TreeNode* node, int key) {
    if (!node) return new TreeNode{key, nullptr, nullptr};
    if (key < node->data) node->left  = insert(node->left,  key);
    else                  node->right = insert(node->right, key);
    return node;
}
// Call as: root = insert(root, key);
```

This style is idiomatic and composable — the "return the new subtree root" pattern extends cleanly to balanced-tree insertions (AVL, red-black) where the recursion has to do rebalancing on the way back up.

Duplicate handling

The pseudocode above treats "equal" as "go right," silently allowing duplicates. For a set-like BST (unique keys), either:

- Short-circuit on equality: return without inserting.
- Treat equality as update: overwrite `cur->data` with the new value (or associated payload).
- Keep a count per node ("multiset" style).

`std::set::insert` returns a pair: the iterator and a bool indicating whether a new element was actually created. That API is exactly "try to insert unique, tell me what happened."

Key idea Insert is the source of the unbalance problem

Search can't make the tree worse — it only reads. Insertion is where tree shape evolves. The sequence of insertions determines the final shape. This is why "random-order insert" and "sorted insert" have such dramatically different costs: they produce different trees. Self-balancing BSTs (AVL, red- black) work by doing extra rotation work *during* insert to keep the tree's shape good regardless of input order.

Aside STL: `std::set::insert` is amortized $O(\log n)$

Red-black insert walks down to the leaf (like a normal BST insert), adds the leaf, then walks back up the path fixing any color-violation from the new red node. The walk up does at most $O(\log n)$ work. Total: $O(\log n)$ worst case, which is better than a raw BST's $O(n)$ worst case.

6 6.6 BST Removal

Removal is the subtle operation. Search is pure inspection, insertion only ever adds leaves, but *removing* a node has to preserve the global ordering invariant on every surviving node. The algorithm splits into three cases depending on how many children the doomed node has.

Definition — The three removal cases

Let X be the node to remove (found by the usual search). Then:

1. X **is a leaf** – unhook it from its parent.
2. X **has exactly one child** – splice the child in where X was.
3. X **has two children** – copy the *in-order successor* (the leftmost node of X's right subtree) into X, then remove that successor from the right subtree.

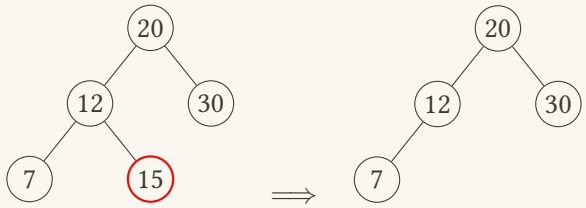
Cases 1 and 2 rewrite one parent pointer. Case 3 is where the cleverness lives.

Why the successor?

The in-order successor is the smallest key in X's right subtree. Copy it over X and every node in X's left subtree is still smaller than the new value there, and every remaining node in X's right subtree is still greater (we removed the minimum of that subtree). The BST invariant is preserved locally, and the successor itself either has no left child (it was the leftmost) or it *is* X's right child directly – so the recursive call is strictly case 1 or case 2. The recursion is one level deep at most.

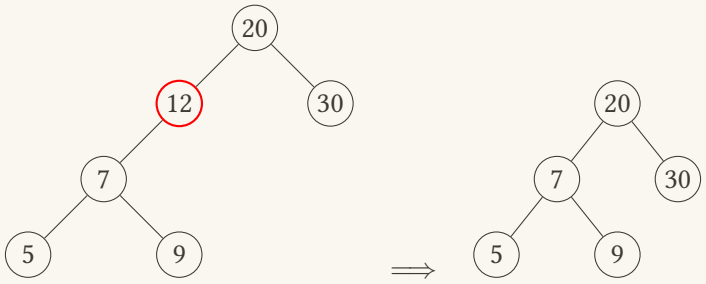
The in-order *predecessor* (rightmost of the left subtree) works just as well; conventions differ. The textbook and the pseudocode here use the successor.

Case 1: removing a leaf



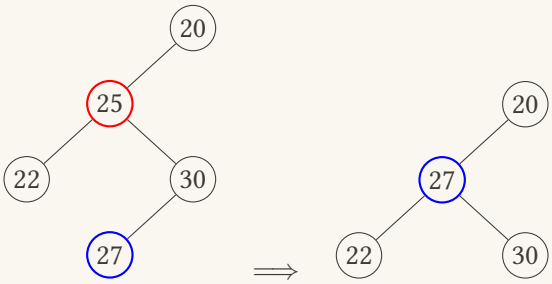
Find parent pointer (12's right), assign `nullptr`, delete the node. If the leaf *is* the root, the tree becomes empty.

Case 2: removing a node with one child



The parent of 12 (here, 20) re-points its left child to 12's only child, 7. The whole subtree below 7 rides along untouched – BST ordering is preserved because everything there was already $< 12 < 20$, which still obeys the grandparent.

Case 3: removing a node with two children



We wanted to delete 25. The successor of 25 is 27 – leftmost node of 25's right subtree (walk right once to 30, then left as far as possible, arriving at 27). Copy 27's key into 25's node, then delete 27 from the right subtree. Because 27 was the *leftmost*, it has no left child, so that recursive delete is a case-1 or case-2 cleanup.

C++ implementation

The reference-to-pointer pattern from insertion pays off again. Each subtree root is passed as `TreeNode*&`, so re-assignment rewires the parent's child slot in place.

```
void removeMin(TreeNode<int>*& root) {
    // Precondition: root != nullptr.
    // Deletes the leftmost node and splices in its right child.
    if (root->left == nullptr) {
        TreeNode<int>* old = root;
        root = root->right;
        delete old;
        return;
    }
    removeMin(root->left);
}

bool bstRemove(TreeNode<int>*& root, int key) {
    if (root == nullptr) return false;

    if (key < root->data) return bstRemove(root->left, key);
    if (key > root->data) return bstRemove(root->right, key);
```

```
    // Found it.
    if (root->left == nullptr) {
        TreeNode<int>* old = root;
        root = root->right;          // Cases 1 and 2 (right-only or leaf).
        delete old;
    } else if (root->right == nullptr) {
        TreeNode<int>* old = root;
        root = root->left;           // Case 2 (left-only).
        delete old;
    } else {
        // Case 3: copy successor, then delete it from the right subtree.
        TreeNode<int>* succ = root->right;
        while (succ->left != nullptr) succ = succ->left;
        root->data = succ->data;
        removeMin(root->right);
    }
    return true;
}
```

Aside

Splitting out `removeMin` is a common refactor – it's exactly the case-3 cleanup step and it's cheaper than a second full `bstRemove` call (no comparisons, just a straight walk left). The textbook pseudocode inlines a recursive call to `BSTRemove`; both are correct.

Iterative bookkeeping: the parent pointer

The recursive version above hides the parent. An iterative version has to track it explicitly:

```
// cur walks down. par trails one step behind so we can rewrite par's child.
TreeNode<int>* par = nullptr;
TreeNode<int>* cur = root;
while (cur != nullptr && cur->data != key) {
    par = cur;
    cur = (key < cur->data) ? cur->left : cur->right;
}
if (cur == nullptr) return;  // Not found.
// ... case split on cur->left / cur->right, rewrite par->left or par->right.
```

This is what the textbook pseudocode does. The recursive `TreeNode*&` trick is cleaner in C++, but the iterative `par`/`cur` pair is universal.

Cost

Operation	Best / balanced	Worst (skewed)
Search to find X	$O(\log N)$	$O(N)$
Find successor (case 3)	$O(\log N)$	$O(N)$
Pointer rewrite	$O(1)$	$O(1)$
Total	$O(\log N)$	$O(N)$

Space is $O(1)$ for the iterative version, $O(h)$ stack for the recursive one. Removal inherits the same

balanced-vs-skewed story as search and insert – the tree's shape dominates everything.

Gotcha

Don't free `cur` before you finish reading `cur->data`, `cur->left`, and `cur->right`. A very common bug: `delete cur; root = cur->right;` reads freed memory. Stash pointers first, delete last.

Key idea

Insertion only adds leaves; removal can touch anywhere. That asymmetry is why a naive BST degrades under churn: if your workload is insert-heavy with sorted-ish input, the tree grows one-sided and never self-corrects. AVL and red-black trees in later sections exist precisely to guarantee $O(\log N)$ *after* arbitrary sequences of inserts and removes.

7 6.7 BST Traversals

A BST stores keys *structurally* sorted – the ordering invariant means a simple recursive walk can visit every node in sorted order. That walk is called an **in-order traversal**, and it's the single biggest reason BSTs earn their keep: you get $O(n)$ sorted output for free, without ever calling a comparison-based sort.

Definition — Tree traversal

A *traversal* visits every node in the tree exactly once, performing some operation (print, count, collect into a list, transform into another structure) on each node. Binary trees have three classical traversal orders, distinguished by when the *current* node is visited relative to its two subtrees.

Three orders, one skeleton

All three traversals share the same shape. They differ only in where the "visit current" step lives:

Order	Pattern	On a BST, this prints…
Pre-order	Cur, Left, Right	keys in root-first order (serialization)
In-order	**Left, Cur, Right**	**keys in ascending sorted order**
Post-order	Left, Right, Cur	keys in leaf-first order (safe for delete)

In-order is what the chapter focuses on. The other two matter later: pre-order is how you serialize or copy a tree, post-order is the deletion order (free the children before the parent).

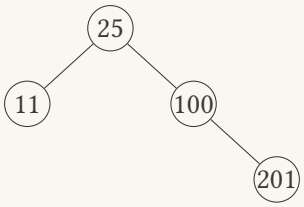
In-order traversal: the algorithm

```
template <typename T>
void bstPrintInorder(const TreeNode<T>* node) {
    if (node == nullptr) return;
    bstPrintInorder(node->left);   // everything smaller than node
    std::cout << node->data << ' '; // node itself
    bstPrintInorder(node->right);  // everything larger than node
}
```

Three lines of logic, after the base case. The BST ordering invariant does the rest: everything in the left subtree is $<$ `node->data`, everything in the right is $>$, so printing them in the order left-cur-right yields ascending sequence.

Walk-through

Trace the in-order traversal of:



1. Call on 25. Descend into left child ⇒ call on 11.
2. Call on 11. Left is null ⇒ return immediately. Print **11**. Right is null ⇒ return.
3. Back at 25. Left finished. Print **25**. Descend into right child ⇒ call on 100.
4. Call on 100. Left is null. Print **100**. Descend into right ⇒ call on 201.
5. Call on 201. Both children null. Print **201**. Return.

Output: `11 25 100 201`. Sorted, as promised.

Key idea

Notice that the nodes are printed *not* in the order the recursion discovers them. The algorithm discovers 25, then 11, then jumps to 100, then 201 – but the *print* happens only after both children have been fully handled on the left side. The call stack is what holds the out-of-order positions until it's their turn.

Why it works: a one-line proof

Induction on tree height. A null tree prints nothing – trivially sorted. Suppose in-order prints every smaller tree in sorted order. For the current tree: by the BST invariant, left-subtree keys are all $<$ `node->data` and right-subtree keys are all $>$. By the induction hypothesis, the left-subtree call prints its contents in sorted order, followed by `node->data`, followed by the right-subtree contents in sorted order. Concatenated, that's the whole tree in sorted order.

Cost

Time	$O(n)$ – each node is visited exactly once
Space	$O(h)$ recursion stack, where h is tree height

The time bound is tight and independent of shape – a skewed tree still traverses in $O(n)$, because you still visit every node once. The *space* bound is where shape matters: a balanced tree uses $O(\log n)$ stack, a degenerate chain uses $O(n)$.

Aside

Since in-order on a BST is sorted, you can implement a "sort an array via BST" algorithm: insert every element, then in-order traverse. Cost: $O(n \log n)$ average if the input is random, $O(n^2)$ worst case if the input is already sorted (insert degrades). This is how tree sort works, and it's why self-balancing trees (AVL, red-black) exist – they make tree sort reliably $O(n \log n)$.

Pre-order and post-order: the sketches

```
template <typename T>
void bstPrintPreorder(const TreeNode<T>* node) {
    if (node == nullptr) return;
    std::cout << node->data << ' ';
    bstPrintPreorder(node->left);
    bstPrintPreorder(node->right);
}

template <typename T>
void bstDestroyPostorder(TreeNode<T>* node) {
    if (node == nullptr) return;
    bstDestroyPostorder(node->left);
    bstDestroyPostorder(node->right);
    delete node;                 // safe: children are already gone
}
```

Pre-order on the example tree above prints `25 11 100 201`; post-order prints `11 201 100 25`. Same nodes, different orders, same $O(n)$ cost.

Gotcha

The in-order output being sorted is a property of **BSTs specifically**. For a generic binary tree with no ordering invariant, in-order just gives "left-cur-right" in structural order – no sortedness. If you find yourself relying on sorted output from in-order, make sure the tree you're walking is actually a BST.

8 6.8 BST Height and Insertion Order

This is the section where the textbook cashes in on all the caveats. We've said "$O(\log n)$ in the balanced case, $O(n)$ in the worst case" for every operation. Now we confront what actually *causes* the worst case, and measure the shape of the tree we end up with.

Bounds on height

For an N-node binary tree, the height h is bounded by:

$$\lfloor \log_2 N \rfloor \;\le\; h \;\le\; N - 1$$

- **Minimum:** $h = \lfloor \log_2 N \rfloor$, achieved when every level is full except possibly the last. This is a *complete* binary tree.
- **Maximum:** $h = N - 1$, achieved when every node has at most one child. This is a *degenerate* tree (a glorified linked list).

Search, insertion, and removal all cost $O(h)$. So the difference between $h = \log_2 N$ and $h = N - 1$ is the difference between a responsive tree on millions of keys and something that stalls.

N	$\log_2 N$	$N - 1$
10	≈ 3	9
1 000	≈ 10	999
10^6	≈ 20	999 999

A million-node balanced BST finds a key in 20 comparisons. A degenerate one takes a million.

What controls the shape? Insertion order.

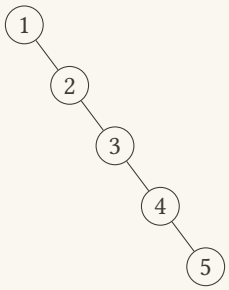
A raw (non-self-balancing) BST has no shape-correcting logic: it just drops new keys wherever the search path leads. So the tree that results from inserting a sequence is *entirely* determined by that sequence.

Definition — The insertion-order rule

- **Random-order inserts:** expected height is $O(\log N)$. In practice the tree stays close to balanced.
- **Sorted (or reverse-sorted) inserts:** height is exactly $N - 1$. Each new key becomes the rightmost (or leftmost) node, forming a chain.
- **Nearly-sorted inserts:** height is $\Theta(N)$. You don't need perfectly sorted input to get worst-case behavior – "mostly sorted" is already disastrous.

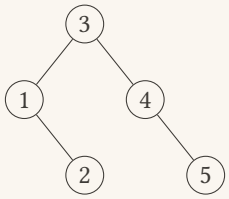
Example: sorted vs. random

Insert $[1, 2, 3, 4, 5]$ in that order into an empty BST:



Height $= 4 = N - 1$. Completely right-skewed. Search for 5 takes 5 comparisons.

Insert the same keys in a randomized order, say $[3, 1, 4, 5, 2]$:



Height $= 2 \approx \lfloor \log_2 5 \rfloor$. Search for 5 takes 3 comparisons. *Same keys, same BST data structure, wildly different performance.*

Key idea

This is the central flaw of the raw BST: performance depends on *the order keys arrived*, not on the keys themselves. A dataset you'd consider "clean" – sorted by user ID, by timestamp, by anything – is exactly the *worst* input for a raw BST. Real-world data is rarely adversarial but frequently sorted-ish.

Computing the height: `bstGetHeight`

Height is defined recursively – perfect match for a recursive traversal:

```
template <typename T>
int bstGetHeight(const TreeNode<T>* node) {
    if (node == nullptr) return -1;   // null subtree has height -1
    int lh = bstGetHeight(node->left);
```

```
    int rh = bstGetHeight(node->right);
    return 1 + std::max(lh, rh);
}
```

The -1 base case is the trick: it makes a single-node tree (both children null) return $1 + \max(-1, -1) = 0$, which matches the definition "a single-node tree has height 0."

Walk-through of `bstGetHeight`

For the tree rooted at 20 with the shape $(20 \;\; (18 \; (12 \; (-, 14), -)) \;\; 30)$:

1. `bstGetHeight(20)` recurses left to 18.
2. `bstGetHeight(18)` recurses left to 12, which recurses left to `null` $\to -1$.
3. `bstGetHeight(12)` now recurses right to 14, which returns $1 + \max(-1, -1) = 0$.
4. `bstGetHeight(12)` returns $1 + \max(-1, 0) = 1$.
5. `bstGetHeight(18)` then recurses right to `null` $\to -1$. Returns $1 + \max(1, -1) = 2$.
6. Back at 20, the right child 30 is a leaf and returns 0. The root returns $1 + \max(2, 0) = 3$.

Tree height $= 3$. Cost of `bstGetHeight`: visits every node once, so $O(n)$ time and $O(h)$ stack.

Cost recap with height in the driver's seat

Operation	Cost	Explanation
`search`, `insert`, `remove`	$O(h)$	one walk down the tree
`inorder traversal`	$O(n)$	visit every node
`bstGetHeight`	$O(n)$	visit every node

For a tree where $h = \log_2 n$, the per-op cost is logarithmic. For $h = n - 1$, it's linear. *Every* BST bound funnels through h.

Aside

If you control the insertion order and the keys are known up front, you can shuffle before inserting. That's a quick fix and often good enough. For streaming inserts or adversarial input, you need a tree that re-balances itself – AVL (6.9) and red-black (6.10) trees do exactly that, paying a constant-factor per-op cost to guarantee $h = O(\log n)$ forever.

Gotcha

Don't confuse "height" with "depth." Height is measured *up from the leaves* to a node (or equivalently, the deepest path in that subtree). Depth is measured *down from the root.* A tree's height equals the maximum depth of any node, but don't conflate the two when computing either one. The algorithm above computes height, not depth.

9 6.9 BST Parent Pointers

Up to this section, our `TreeNode` has held only `data`, `left`, and `right`. Every algorithm has propagated "who is my parent?" information down through recursion or through a trailing `par` pointer in iterative code. That works, but it means *going up the tree always requires re-descending from the root.* Self-balancing trees (AVL, red-black) rebalance after a modification, and the rebalancing always starts at the modified node and walks *upward* – they need $O(1)$ access to the parent.

The fix is a third pointer per node.

Definition — Parent pointer

Each `TreeNode` gets an additional field `parent` – a pointer to the node whose `left` or `right` points to this one. The root has `parent = nullptr`. The invariant to maintain: for every non-null node n, `n->parent->left == n` **or** `n->parent->right == n`, and the root has no parent.

```
template <typename T>
struct TreeNode {
    T          data;
    TreeNode*  left   = nullptr;
    TreeNode*  right  = nullptr;
    TreeNode*  parent = nullptr;   // new
};
```

What the extra pointer buys you

- $O(1)$ parent lookup from any node.
- $O(1)$ sibling lookup (if `node == node->parent->left` the sibling is `node->parent->right`, else `node->parent->left`).
- Walking upward until a rotation point is found – essential for AVL's retrace step and red-black's fix-up.
- Iterator-style traversals (give me the next in-order node) can be implemented without a stack.

The cost: one extra pointer per node (same memory cost as a doubly-linked list over a singly-linked list), and *every* insertion and removal must now maintain the parent-pointer invariant.

Insert with parent wiring

The search walk is unchanged. The difference is that when we finally attach the new node, we wire up `new->parent` to the node we attached below:

```
template <typename T>
void bstInsert(TreeNode<T>*& root, TreeNode<T>* node) {
    node->left = node->right = node->parent = nullptr;

    if (root == nullptr) { root = node; return; }

    TreeNode<T>* cur = root;
    while (true) {
        if (node->data < cur->data) {
            if (cur->left == nullptr) {
                cur->left    = node;
```

```
                node->parent = cur;     // wire the back-pointer
                return;
            }
            cur = cur->left;
        } else {
            if (cur->right == nullptr) {
                cur->right   = node;
                node->parent = cur;     // wire the back-pointer
                return;
            }
            cur = cur->right;
        }
    }
}
```

A tiny helper: `bstReplaceChild`

Removal with parent pointers becomes much cleaner if we factor out the "swap this child out for that one" operation. Because any node that isn't the root knows its parent, we can write:

```
template <typename T>
bool bstReplaceChild(TreeNode<T>* parent,
                     TreeNode<T>* currentChild,
                     TreeNode<T>* newChild) {
    if (parent->left != currentChild && parent->right != currentChild)
        return false;

    if (parent->left == currentChild) parent->left = newChild;
    else                              parent->right = newChild;

    if (newChild != nullptr) newChild->parent = parent;
    return true;
}
```

This helper guarantees two things at once: the parent's `left`/`right` slot gets the new child, *and* the new child's `parent` back-pointer gets updated. Without this helper you will eventually write code that updates one direction and forgets the other – that's the single most common parent-pointer bug.

Removal, redone with parent pointers

Splitting removal into two functions is idiomatic once you have parent pointers: the outer `bstRemoveKey` finds the node by key, the inner `bstRemoveNode` removes a known node. The inner version is the one that AVL and red-black trees will override to hook in rebalancing.

```
template <typename T>
void bstRemoveKey(TreeNode<T>*& root, const T& key) {
    TreeNode<T>* node = bstSearch(root, key);
    bstRemoveNode(root, node);
}

template <typename T>
void bstRemoveNode(TreeNode<T>*& root, TreeNode<T>* node) {
```

```
    if (node == nullptr) return;

    // Case 1: two children -- copy successor, recurse to remove it.
    if (node->left != nullptr && node->right != nullptr) {
        TreeNode<T>* succ = node->right;
        while (succ->left != nullptr) succ = succ->left;
        node->data = succ->data;
        bstRemoveNode(root, succ);
        return;
    }

    // One or zero children from here on.
    TreeNode<T>* only = (node->left != nullptr) ? node->left : node->right;

    // Case 2: the node is the root.
    if (node == root) {
        root = only;
        if (root != nullptr) root->parent = nullptr;
    }
    // Case 3 & 4: splice the only (possibly-null) child in below node->parent
        .
    else {
        bstReplaceChild(node->parent, node, only);
    }

    delete node;
}
```

Compare this to the reference-to-pointer version from 6.6. Both are correct; the parent-pointer version has four clean cases that mirror the textbook's breakdown, and the `bstReplaceChild` helper keeps the back-pointer invariant airtight.

Worked case analysis

Case	Condition	Action
1	two children	Copy successor's data, recurse on successor
2	is the root	Promote lone child (or null) as the new root
3	only left child	`bstReplaceChild(parent, node, node->left)`
4	only right / none	`bstReplaceChild(parent, node, node->right)`

Case 1 recursing into the successor is safe: the successor has at most one child (it was the leftmost of the right subtree), so the recursive call falls into case 2, 3, or 4.

Key idea

The parent pointer is a trade: one extra pointer per node and careful wiring on every mutation, in exchange for $O(1)$ upward traversal. For a raw BST that only does search/insert/remove, it's not worth it – descent-based code gets the same information from the recursion stack. For a self-balancing tree, it's *essential* – AVL's retrace walks upward from the modified leaf, and red-black's fix-up walks upward from the newly-inserted node looking for violated constraints.

The wiring traps

Gotcha

The invariant "every non-root node has a parent whose child-slot points back at it" is easy to state but easy to break:

- Forgetting to set `new->parent` when inserting.
- Forgetting to `null` the old node's parent pointer when promoting a child to root.
- Copying nodes (by value!) in case 1 – the successor's *data* is copied, not the node. If you accidentally re-wire around the successor's node instead, you leave dangling parent pointers.
- `bstReplaceChild` with `newChild = nullptr` is legal (that's how leaves are removed), but you must skip the `newChild->parent = parent` assignment in that case.

Using `bstReplaceChild` everywhere instead of hand-rolling the assignment eliminates most of these bugs.

10 6.10 Recursive BST Implementations (No Parent Pointers)

This section closes the chapter by revisiting the core BST operations – search, find-parent, insert, remove – as *recursive* algorithms on a plain BST without parent pointers. The point isn't to pick a winner between iterative and recursive: both are correct and produce identical trees. The point is to see how the tree-shaped data structure wants to be written tree-shaped, and to recognize the patterns you'll need again for AVL, red-black, tries, and every other hierarchical structure in your future.

Key idea

The recursion mirrors the structure of the data. Every non-trivial BST problem has the shape "do something at the root, recurse into the appropriate subtree." That symmetry is why recursion often produces the shortest, clearest code for trees. The recursion stack plays the role of the `parent` pointer chain from section 6.9 – going "up" is just returning from a recursive call.

Recursive search

The recursive version of search is almost a direct transcription of the ordering invariant:

```
template <typename T>
const TreeNode<T>* bstSearch(const TreeNode<T>* node, const T& key) {
    if (node == nullptr)       return nullptr;       // base: not found
    if (key == node->data)     return node;          // base: found
    if (key < node->data)      return bstSearch(node->left,  key);
    return                            bstSearch(node->right, key);
}
```

Two base cases, two recursive cases. Each call does $O(1)$ work and chooses exactly one child, so the total cost is $O(h)$ – same as iterative. The difference is purely syntactic. The recursive version uses $O(h)$ stack rather than $O(1)$, so on a pathological chain you can blow the stack – but that's the same input that would make any BST operation slow, so it's rarely the binding concern.

Aside

Each of the three recursive calls is in *tail position* (the last thing the function does). A smart compiler can optimize this to a loop (tail-call elimination), giving you iterative performance from recursive code. GCC and Clang do this reliably at `-O2`. You don't have to manually iterativize search for perf reasons – write it however is clearest, measure if you care.

Recursive find-parent (without parent pointers)

Section 6.9 added a `parent` pointer to each node to make "who is my parent?" a $O(1)$ lookup. Without that field, you have to walk from the root again. The algorithm is structurally identical to `bstSearch` – just with a different base case.

```
template <typename T>
TreeNode<T>* bstGetParent(TreeNode<T>* subtreeRoot, const TreeNode<T>* node) {
    if (subtreeRoot == nullptr) return nullptr;
    if (subtreeRoot->left == node || subtreeRoot->right == node)
        return subtreeRoot;                          // base: found the parent
    if (node->data < subtreeRoot->data)
        return bstGetParent(subtreeRoot->left,  node);
    return bstGetParent(subtreeRoot->right, node);
}
```

Notice the base case compares *pointers*, not keys: `subtreeRoot->left == node`. Key comparison would be wrong if the tree contained duplicates. Pointer comparison asks "is the node I'm looking for literally this one's child?"

Cost: $O(h)$. Same as search, because the decision of which subtree to recurse into is made with the same key comparison.

Recursive insertion

The recursive insert descends until it finds a null child slot, then attaches the new node:

```
template <typename T>
void bstInsertRec(TreeNode<T>* parent, TreeNode<T>* nodeToInsert) {
    if (nodeToInsert->data < parent->data) {
        if (parent->left == nullptr) parent->left = nodeToInsert;
        else bstInsertRec(parent->left, nodeToInsert);
    } else {
        if (parent->right == nullptr) parent->right = nodeToInsert;
        else bstInsertRec(parent->right, nodeToInsert);
    }
}

template <typename T>
void bstInsert(TreeNode<T>*& root, TreeNode<T>* node) {
    if (root == nullptr) root = node;
    else                 bstInsertRec(root, node);
}
```

Compare this with the reference-to-pointer version from 6.5 (`TreeNode*&` everywhere). That version is

arguably more elegant – it handles the empty-tree case without a wrapper, because the reference to the caller's pointer *is* where we write. The `parent`-passing version here has to special-case the empty root in the outer wrapper. Both are valid; the `TreeNode*&` version is tighter C++, the `parent`-passing version translates more directly to languages without reference semantics (Java, Python, Go).

Recursive removal

Removal combines `bstSearch` (to find the node) and `bstGetParent` (to find its parent) and then applies the four cases from section 6.6:

```
template <typename T>
bool bstRemoveNode(TreeNode<T>*& root,
                   TreeNode<T>* parent,
                   TreeNode<T>* node) {
    if (node == nullptr) return false;

    // Case 1: two children. Find successor + its parent, copy, recurse.
    if (node->left != nullptr && node->right != nullptr) {
        TreeNode<T>* succ       = node->right;
        TreeNode<T>* succParent = node;
        while (succ->left != nullptr) {
            succParent = succ;
            succ       = succ->left;
        }
        node->data = succ->data;
        return bstRemoveNode(root, succParent, succ);
    }

    TreeNode<T>* only = (node->left != nullptr) ? node->left : node->right;

    // Case 2: node is the root.
    if (node == root) {
        root = only;
    }
    // Cases 3 & 4: splice below parent.
    else if (parent->left == node) parent->left  = only;
    else                           parent->right = only;

    delete node;
    return true;
}

template <typename T>
bool bstRemove(TreeNode<T>*& root, const T& key) {
    TreeNode<T>* node   = const_cast<TreeNode<T>*>(bstSearch(root, key));
    TreeNode<T>* parent = bstGetParent(root, node);
    return bstRemoveNode(root, parent, node);
}
```

This is the same four-case removal as before. The key implementation difference is that `succParent` is tracked explicitly during the successor walk – since we have no parent pointers, we can't ask the successor for its parent, so we maintain the trailing pointer manually (exactly the `par`/`cur` trick from iterative search).

Comparison: three ways to remove a node

Version	Extra state per node	Cost of finding parent
`TreeNode*&` reference (6.6)	none	free (held in the recursion)
parent pointers (6.9)	+1 pointer	$O(1)$
plain recursive + `bstGetParent` (6.10)	none	$O(h)$ (re-search)

The recursive version with `bstGetParent` is the slowest of the three for removal – you pay $O(h)$ once for search, then again for `bstGetParent`. It's still $O(h)$ total asymptotically, but the constant doubles. The `TreeNode*&` version is the tightest for a raw BST. The parent-pointer version is what real libraries use when they're building AVL or red-black on top.

Key idea

The three implementations are *equivalent in behavior* – they produce the same trees from the same operations. They differ only in how parent information is retrieved: implicitly via the recursion stack, explicitly via a stored pointer, or on demand by re-searching. Choose based on what else the structure needs. A raw BST: reference-to-pointer. A self-balancing tree: parent pointers. A teaching example where you want two operations to feel symmetric: plain recursion with `bstGetParent`.

Gotcha

Watch the succession walk in case 1 carefully. The `while (succ->left != nullptr)` loop walks *through* the successor's ancestors, updating `succParent` each step. If you initialize `succParent = nullptr` and forget that `node->right` itself might be the successor (when its left is null), you'll end up with `succParent = nullptr` at case-1's recursive call, and the recursion will mis-handle the splice. Initialize `succParent = node`.

Aside

Chapter 6 ends here. The next chapters extend these foundations to self-balancing trees (AVL and red-black), heaps (array-backed complete trees with the partial-order invariant), and general trees / tries. Everything we've built – recursion on subtrees, the left-root-right pattern, the four-case removal – will reappear, sometimes unchanged, sometimes augmented with rebalancing rotations. The shape is stable; the invariants get stricter.

What this connects to

Aside Forward links to later chapters

- **Chapters 7-8 (AVL, red-black).** These make the BST $O(\log n)$ guarantee tight instead of best-case. Every rotation preserves the BST invariant from this chapter; only the balance invariant is new. If BST insert/remove is shaky, AVL won't land.
- **Optional ch. 11 (B-trees / 2-3-4 trees).** Generalizes "one key per node, two children" to "multiple keys, multiple children." Used by on-disk databases and filesystems.
- **`std::map` / `std::set`.** Both are red-black trees under the hood. Choosing between them and their `unordered_` hash-based cousins is the ADT-vs-data-structure decision from ch. 3, now grounded in a real cost model.
- **Heaps and priority queues.** Heaps are another hierarchical structure, but instead of BST's

key-ordering invariant they use a *partial-order* (parent $\leq$ child) invariant. Same node + children idea, different constraint.

- **Graphs (opt. ch. 10).** Graph traversal generalizes tree traversal: a tree is a graph without cycles. BFS and DFS on graphs reuse the queue / stack idioms from ch. 4 and the recursion pattern from here.

Companion materials. One-page cheat sheet: `cheat_sheets/ch_6.tex`. Practice prompts (problem $\rightarrow$ pseudocode $\rightarrow$ C++): `practice_prompts/ch_6.md`.